Efficient Attention Mechanisms for Large Language Models: A Survey

Yutao Sun*, **Zhenyu Li***, **Yike Zhang***, **Tengyu Pan***, **Bowen Dong***,
Yuyi Guo, Jianyong Wang†

Tsinghua University

arXiv:2507.19595v2 [cs.CL] 7 Aug 2025

Abstract

Transformer-based architectures have become the prevailing backbone of large language models. However, the quadratic time and memory complexity of self-attention remains a fundamental obstacle to efficient long-context modeling. To address this limitation, recent research has introduced two principal categories of efficient attention mechanisms. Linear attention methods achieve linear complexity through kernel approximations, recurrent formulations, or fast-weight dynamics, thereby enabling scalable inference with reduced computational overhead. Sparse attention techniques, in contrast, limit attention computation to selected subsets of tokens based on fixed patterns, block-wise routing, or clustering strategies, enhancing efficiency while preserving contextual coverage. This survey provides a systematic and comprehensive overview of these developments, integrating both algorithmic innovations and hardware-level considerations. In addition, we analyze the incorporation of efficient attention into large-scale pre-trained language models, including both architectures built entirely on efficient attention and hybrid designs that combine local and global components. By aligning theoretical foundations with practical deployment strategies, this work aims to serve as a foundational reference for advancing the design of scalable and efficient language models.

Keywords: Language Model, Linear Attention, Sparse Attention, Hybrid Model

* Equal Contribution. † Corresponding Author(s).

Contents

1 Introduction **3**

2 Linear Attention **4**

2.1 Kernelized Linear Attention 4

2.2 Linear Attention with Forgetting Mechanism 5

2.2.1 Data-Independent Decay 6

2.2.2 Data-Dependent Decay 6

2.3 Linear Attention as In-Context Learners 7

2.3.1 Learning Objective 8

2.4 Discussion on Other Designs 9

2.4.1 Element-wise Linear Attention 9

2.4.2 Multi-Pass Linear Attention 10

2.4.3 Bidirectional Linear Attention 10

2.5 Hardware Implementation 10

3 Sparse Attention **12**

3.1 Fixed-pattern Sparse Attention 12

3.2 Block Sparse Attention 13

3.2.1 Block-Sparse Attention for Prefill 13

3.2.2 Block-Sparse Attention for Decode 14

3.2.3 Routing-based Block-Sparse Attention 15

3.2.4 System-level Design Choices 16

3.3 Clustering Attention 16

3.4 Bidirectional Sparse Attention 17

4 Pretrained LLM with Efficient Attention **17**

4.1 Pretrained Models with Uniform Efficient Attention 17

4.2 Pretrained Models with Hybrid Efficient Attention 18

5 Outlook **19**

6 Acknowledgment **20**

1. Introduction

Transformer-based architectures [60] have become the de-facto choice as the backbone of modern Large Language Models (LLMs). Despite their success, the standard self-attention mechanism remains a significant computational bottleneck, with quadratic time and memory complexity with respect to input sequence length. This limitation poses substantial challenges for scaling LLMs to handle increasingly long contexts with both strong performance and high efficiency.

To address this, two major directions have emerged to reduce the time and space complexity of softmax Attention. The first mechanism is Linear Attention [17, 22, 49, 52, 69, 71], which seeks to reduce attention complexity by reparameterizing or approximating the softmax attention as linear operations. The second candidate is Sparse Attention [8, 21, 33, 55, 72], which restricts attention computation to a subset of the full key space based on fixed or dynamic sparsity patterns. While both approaches aim to improve efficiency, they differ significantly in formulation, design choices, and hardware implications.

This survey provides a comprehensive review of recent developments in Efficient Attention mechanisms, with a dual focus on algorithmic principles and system-level implementation. Based on that, we also study the Pre-trained LLM employing these Efficient Attentions.

We categorize linear attention methods into three major paradigms. First, kernelized linear attention approximates the softmax kernel with inner products in a feature space, achieving linear complexity via random feature maps [9, 41] or fixed positive mappings [22]. Second, recurrent linear attention with forgetting mechanisms introduces position-aware recurrence, enabling long-sequence modeling through data-independent [52] or data-dependent decay [17, 69], which control how past information fades over time. Third, fast-weight and meta-learning-based formulations reinterpret linear attention as a memory update process optimized online, where models such as DeltaNet [49, 71] and TTT [50, 51] incorporate fast-learning dynamics directly into the state evolution. We also examine hardware-friendly representations of linear attention—including parallel, recurrent, and chunkwise forms—highlighting their respective trade-offs in computational complexity, memory footprint, and compatibility with training or inference workflows.

We classify Sparse Attention into fixed-pattern sparsity, block sparsity, and clustering-based sparsity. Fixed-pattern sparsity adopts static token-level masks such as sliding windows, dilated positions, or designated global tokens, offering simplicity and hardware-friendliness [4, 8, 14, 64]. Block sparsity selects or routes attention at block granularity, either via heuristic scoring [33, 55, 65], trainable gating [15, 72], enabling structured memory access and efficient GPU utilization. Clustering-based sparsity organizes key-value pairs using content-based or position-aware grouping methods such as k-means or LSH, facilitating semantically aware retrieval with reduced memory overhead [7, 24, 31]. Finally, we also discuss bidirectional sparse designs extend sparsity patterns to encoder-style models. These approaches differ in sparsity granularity, selection mechanism, and their alignment with hardware primitives like FlashAttention [10], and collectively represent the foundation for efficient long-context modeling in modern Transformers.

There are recent efforts to integrate efficient attention mechanisms into industry-level Pre-trained Language Models. These include both pure efficient architectures—such as linear attention and state-space models, and hybrid designs that combine local and global attention patterns. Models like EAGLE [38], Falcon Mamba [78] and MiniCPM4 [57] demonstrate the scalability of purely linear or sparse approaches to the multi-billion parameter scale, offering strong performance with constant-time inference. Meanwhile, hybrid models [5, 26, 28, 32, 53, 56, 66]

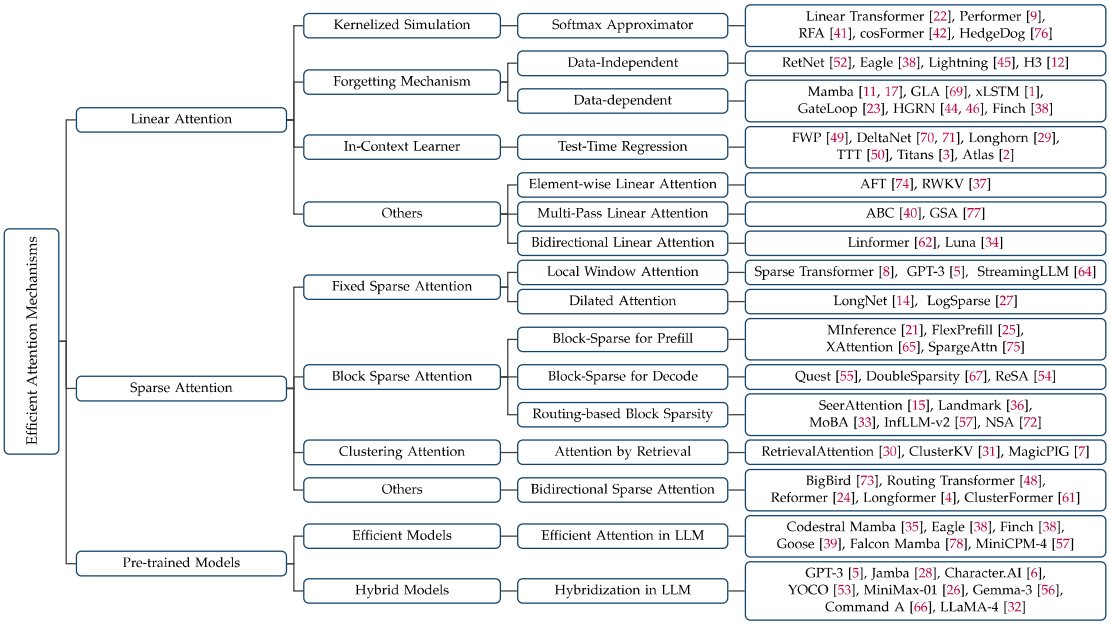


Figure 1. Taxonomy of Efficient Attention Mechanisms.

interleave dense, sparse, and local attention to balance computational efficiency with context modeling capacity, reflecting a growing trend toward compositional, hardware-aware attention designs in modern LLMs.

Our goal is to provide a unified framework for understanding the evolution of attention mechanisms under both algorithmic and hardware constraints, and how these designs are integrated into scalable LLM architectures. By connecting theoretical insights with practical implementations, we hope this survey offers a valuable reference for researchers and practitioners working toward efficient and deployable model design.

To structure this survey, we organize the discussion as follows:

- Section 2 introduces Linear Attention, covering its evolution across different model generations, the associated design principles, and implications for hardware implementation.
- Section 3 presents Sparse Attention, categorizing sparsity patterns, analyzing deployment scenarios, and offering practical system-level design recommendations.
- Section 4 reviews Pre-trained Language Models that incorporate efficient attention mechanisms, including both uniform efficient architectures and hybrid models that integrate local, sparse, and dense attention.
- Section 5 offers an Outlook on future directions, discussing open challenges and potential advances in algorithmic and hardware-aligned research.

2. Linear Attention

2.1. Kernelized Linear Attention

Traditional linear attention methods seek to approximate the softmax-based attention mechanism in a way that scales linearly with sequence length. The core idea is to replace the expensive

softmax computation with a *kernel-based* approximation of the attention weights. In standard self-attention, each output is a weighted sum of values V with weights given by a softmax over query-key similarities:

$$\text{Attn}(Q, K, V) = \text{softmax}(QK^\top)V, \tag{1}$$

where $Q, K, V \in \mathbb{R}^{L\times d}$ (with L the sequence length and d the model dimension per head). The softmax yields weights $\propto \exp(q_i^\top k_j)$ for query q_i and key k_j. Kernelized Linear attention instead finds a feature mapping $\phi(\cdot)$ such that the softmax kernel is approximated by a simple dot-product in an induced feature space: $\exp(q^\top k) \approx \phi(q)^\top \phi(k)$ [59]. Given such a ϕ, one can rewrite attention as:

$$O = \frac{\phi(Q)(\phi(K)^\top V)}{\phi(Q)(\phi(K)^\top \mathbf{1})} \tag{2}$$

$\phi(\cdot)$ is usually chosen to produce non-negative outputs since $\exp(\cdot)$'s value region is non-negative, a normalization divisor is also applied to mimic the softmax probabilities. This reformulation reduces complexity from $O(L^2 d)$ to $O(Ld^2)$ (or even $O(Ld)$ with suitable feature dimension reduction), since the expensive $L \times L$ attention matrix is never formed explicitly.

Linear Tranformer [22] replaces the softmax kernel with a fixed positive feature map. In practice they set $\phi(x) = \text{ELU}(x) + 1$. $\text{ELU}(\cdot)$ is differentiable in the whole defined region, showing a better performance with naive $\text{ReLU}(\cdot)$ function.

Performer [9] introduces FAVOR+ – a Random Features scheme that unbiasedly approximates the softmax kernel. It samples randomized feature maps ϕ so that $E[\phi(Q)\phi(K)^\top] = \exp(QK\top)$. This yields a provably unbiased estimator of full softmax attention using only O(N) operations. In particular, Performers use positive orthogonal random features, which reduce variance in the approximation.

Random Feature Attention [41] is a linear attention built via Random Fourier Features for the softmax kernel. Similar to Performer, RFA leverages random mapping and triangular activation to approximate softmax. RFA further normalizes the queries and keys before random projection to reduce variance. RFA also has a variant, RFA-Gate, which adds an optional gating mechanism for recency bias.

cosFormer [42] proposes to use cosine function to approximate softmax. Since $\cos(a + b) = \cos a \cos b - \sin a \sin b$, cosFormer decomposes the cosine re-weighted attention $S_{ij} = Q'_i K'_j \cos(\frac{\pi}{2} \times \frac{i-j}{M})$ into a linear attention form.

HedgeDog [76] leverages a spiky kernel $\phi(x) = \exp(Wx + b)$ since they observe that the performance gap between Transformer and Linear Transformer is due to the lack of spiky and monotonic properties. HedgeDog shows a better attention entropy and monotonicity.

2.2. Linear Attention with Forgetting Mechanism

A more recent line of work interprets attention through the lens of recurrent neural networks or continuous state-space models. While Traditional Linear Attention is usually position unaware, where the recurrence order does not make influence on the output, modern Linear Attention behaves more like RNNs with state-tracking and hidden memory. Therefore, these models explicitly incorporate recurrence, gating, or state dynamics to handle long sequences with linear complexity. Decay factor is the most important factor to bring forgettimg mechanism.

2.2.1. *Data-Independent Decay*

Retentive Networks (RetNet) [52] introduce a *retention* mechanism that replaces attention with a recurrent-style update using fixed decay coefficients. In a RetNet layer, each time step t maintains a state vector s_t that aggregates past inputs with exponential forgetting. The recurrence can be written as

$$S_t = \gamma S_{t-1} + k_t^\top v_t \tag{3}$$

with $\gamma \in (0, 1)$ a learned decay factor (per retention head) and $k_t^\top v_t$ a new contribution from the current token (v_t is a value projection of x_t, and k_t is a key projection). The output is then obtained by a linear "query" projection: $o_t = q_t, s_t$. Unrolling equation 3 gives an explicit formula for retention:

$$o_t = q_t S_t = \sum_{n=1}^{t} \gamma^{t-n} q_t k_t^\top v_t \tag{4}$$

which shows that contributions from token n are exponentially decayed by the factor γ^{t-n} by the time step t. Crucially, γ is a *data-independent* decay, which is a fixed parameter for the layer (often one per head in multi-head retention), not a function of the input content. It endows RetNet with an O(1) memory update like an RNN, while still allowing parallel computation during training via an equivalent matrix formulation. (For example, one can show that equation 3 is equivalent to a "retention matrix" form , $\text{Retention}(X) = (QK^\top \odot D), V$, where $D_{t,n} = \gamma^{t-n}$ for $t \geq n$ implements the decay and causal masking.)

RetNet's retention mechanism shares themes with other data-independent recurrence models.

Eagle Eagle [38] improves RWKV design with outer-product memory, which is equivalent to Linear Attention. In RWKV series, the decay factor is parameterized as $\gamma = \exp(-\exp(w))$ where w is a data-independent learnable factor. In practice, both RetNet and Eagle achieve linear inference scaling with competitive performance, using fixed decays to forget old information. Empirically, RetNet uses a fixed scalar γ per head (often each layer has multiple retention heads with different γ values, giving a form of multi-scale decay), whereas Eagle uses learnable scalar w to parameterize decay factor.

Lightning Attention [43, 45] also proposes a linear attention layer augmented with a fixed scalar decay per head to achieve length-invariant computation speed. In Lightning Attention, the hidden state is essentially $s_t = \lambda s_{t-1} + k_t^\top v_t$ for some constant λ (with λ learned or set by the model), which is in the same spirit as RetNet's γ but optimized for hardware efficiency.

H3 [12] introduces a recurrent state-space model [18] into linear attention, using learned, data-independent exponential decay via SSM for key-value outer-product hidden state. While Linear Attention enables efficient training via chunk-wise computation, H3 requires explicit state expansion for SSM computation, thus restricts the head dimension contributing to limited expressiveness.

In summary, data-independent decay methods maintain a persistent state that fades over time at a predetermined rate, enabling $O(1)$ recurrence and constant memory per step. They sacrifice some adaptability, which motivates the introduction of data-dependent mechanisms in more recent models.

2.2.2. *Data-Dependent Decay*

While fixed decays offer simplicity and speed, they may under-utilize information from the input stream. Gated or data-dependent approaches make the forgetting factor itself a learned

function of the current input. The general form of such a recurrent update is:

$$S_t = G_t S_{t-1} + k_t^\top v_t \tag{5}$$

where S_{t-1} is the previous state, and G_t is a gating tensor determined by the token x_t . If G_t is close to 0 in some component, the past state in that component is largely forgotten at time t; if $G_t \approx 1$, the past is retained. Unlike the constant γ in RetNet, here G_t varies with t via x_t. Two notable examples of this strategy in large language model design are Mamba [11, 17] and Gated Linear Attention (GLA) [69].

Mamba is a recurrent state-space model that endows the state decay rates with input dependence. In each Mamba layer, the base state evolution is similar to S4 [18], but the state matrix is effectively made dynamic. G_t is a group-wise vector ranging from 0 to 1 as a dynamic forget gate. Which bridges a gap between attention and pure SSMs. Empirical results show that Mamba2 can outperform Transformers of similar or even larger size on language modeling tasks, highlighting the power of data-dependent decay in long-sequence modeling.

GLA directly introduces gating mechanism in the Linear Attention, where a gating function is embedded into a linearized attention layer to improve its expressiveness. GLA modifies retention recurrence by a learnable element-wise forget gate G_t.

Beyond these, several other models similarly endow their recurrence with content-dependent gates.

xLSTM [1] replaces the standard sigmoid forget gate with an exponential transform of linear gate signals (with normalization), yielding a smooth, input-conditioned decay on its cell state.

GateLoop [23] applies head-wise gate based on retention, which enables a simple but effective data-dependent decay while maintaining efficient hardware implementation.

HGRN [44] introduces gated recurrence in the linear RNNs. HGRN2 [46] further add state expansion into HGRN framework. State expansion is equivalent to key-value out product in Linear Attention.

Finch [38] employs data-dependent gating on Eagle. Since Eagle is similar to Retention with other orthogonal modifications, Finch also shows deep connection with above models.

In summary, data-dependent decay models augment linear attention or RNN-style architectures with content-based gates that control the flow of information. The results in the paper show that these models can often match or exceed Transformer performance on language tasks, while scaling to very long inputs.

2.3. Linear Attention as In-Context Learners

Beyond the efficiency gains offered by linear attention mechanisms, a significant advancement lies in their application to enhance *in-context learning*. This refers to the capability of a model to rapidly adapt or learn from a given prompt without requiring explicit gradient updates to its pre-trained weights.

While large Transformer models inherently exhibit in-context learning by interpreting the prompt as a form of training data, recent innovations have integrated fast learning rules directly into the attention mechanism, effectively treating sequence processing as an online training process. **FWP** [49] establish a formal equivalence between existing linear attention mechanisms and Fast Weight Programmers. In the FWP paradigm, a slow neural network learns to program the "fast weights" of another network, often via additive outer products of self-invented key and

value patterns. This section explores several models, including DeltaNet [70, 71], Longhorn [29], Test-Time Training (TTT) layers [50, 51], Titans [3], that exemplify this paradigm of leveraging linear attention as an in-context learner through mechanisms like fast weight updates, viewed through a meta-learning lens.

Method	Update Rule
Linear Attention [22]	$S_t = S_{t-1} + k_t v_t^\top$
RetNet [52]	$S_t = \gamma S_{t-1} + k_t v_t^\top$
GLA [69]	$S_t = S_{t-1}\text{Diag}(a_t) + k_t v_t^\top$
Mamba [17]	$S_t = \alpha_t S_{t-1} + b_t k_t v_t^\top$
HGRN-2 [46]	$S_t = S_{t-1}\text{Diag}(a_t) + (1-a_t) v_t^\top$
DeltaNet [71]	$S_t = S_{t-1}(1-\beta_t k_t k_t^\top) + \beta_t k_t v_t^\top$
Gated DeltaNet [70]	$S_t = \alpha_t S_{t-1}(1-\beta_t k_t k_t^\top) + \beta_t k_t v_t^\top$
TTT [51]	$S_t = S_{t-1} - \eta_t \nabla_S \ell(S_{t-1}; k_t, v_t)$

Table 1. Update rule among different Linear Attention variants. Each model is a recurrence on matrix memory S_t.

2.3.1. Learning Objective

From a meta-learning perspective, these models define an implicit learning objective that is optimized during inference. Denote q_t, k_t, v_t as the query, key and value at timestep t, the context memory S_t is optimized by the following objective:

$$\mathcal{L}_t(S) = \frac{1}{2}||f_S(k_t) - v_t||^2 \tag{6}$$

DeltaNet incorporates the classical delta rule [49], where $f_S(k_t) = Sk_t$. Its update rule will be $S_t = S_{t-1} + \eta_t(v_t - S_{t-1}k_t)k_t^\top$, can be derived by minimizing the error between the current memory retrieval $S_{t-1}k_t$ and the new value v_t. This signifies a step towards learning the key-value mapping online, effectively refining the memory based on the immediate context.

TTT [51] generalizes the meta-learning objective with different modeling architecture:

$$f_S(k_t) = \begin{cases} \text{LN}(S\,k_t) + k_t, & \text{TTT-Linear} \\ \text{LN}\big(\text{MLP}_S(k_t)\big) + k_t, & \text{TTT-MLP} \end{cases} \tag{7}$$

The context network f_S enhances the capability of in-context meta-learning. However, since the gradient of f_S is much more complicated than as a simple linear projection, the online update can not be written as a simple rule.

Batch Update Batch update tries to solve the difficulties of the training parallelism when f_S works as a neural network. Usually, context memory is meta-learned with a batch size of 1, which is not feasible for general TTT models. Instead, analogous to chunk parallelism, TTT treats an entire chunk as a batch. There is no state updates occur within the batch (i.e., S remains constant). After processing the batch, S is updated once using the aggregated gradients or update signals from all samples in the batch. This strategy preserves parallel efficiency while accommodating the training requirements of more complex architectures.

Momentum **Titans** [3] introduces momentum, which is commonly used in optimization, to strengthen the the capability of the memory update mechanism:

$$\begin{aligned} \mathcal{M}_t &= (1-\alpha_t)\mathcal{M}_{t-1} + S_t \\ o_t &= q_t \mathcal{M}_t \end{aligned} \tag{8}$$

The momentum term allows the memory to accumulate information gradually with an exponetial moving average on the state S. This can be seen as a form of meta-learning where the update rule itself learns to be more stable and robust over long sequences.

Weight Decay Weight decay is another regularization technique in training, corresponding the forgetting mechanism within Linear Attention models. **Gated DeltaNet** [70] and Titans employs weight decay in its memory update, serving as a learned forget gate to limit the influence of very old or noisy data. It corresponds to the selective state retention mechanisms found in architectures like RetNet [52] and Mamba [17], where the decay mechanism is proven crucial for language modeling performance:

$$S_n = \gamma_n S_{t-1} + \eta_t (v_t - S_{t-1} k_t) k_t^\top \tag{9}$$

In summary, these advancements in linear attention mechanisms are pushing the boundaries of in-context learning by explicitly incorporating meta-learning principles into their architecture. Through fast weight updates, sophisticated memory management techniques, and online learning rules, these models are moving towards a paradigm where the distinction between training and inference becomes increasingly blurred, leading to more efficient and adaptable large language models capable of learning and leveraging knowledge directly from the context.

2.4. Discussion on Other Designs

2.4.1. Element-wise Linear Attention

Attention-Free Transformer [74] leverages a simple weight $\exp(K_{t'} + w_{t,t'})$ instead of $\exp(QK^\top)$:

$$O_t = \sigma_q(Q_t) \odot \frac{\sum_{t'=1}^{t} \exp(K_{t'} + w_{t,t'}) \odot V_{t'}}{\sum_{t'=1}^{t} \exp(K_{t'} + w_{t,t'})} \tag{10}$$

Where $w_{t,t'}$ is learned pair-wise position biases. Among the AFT variants, AFT-Simple remove $w_{t,t'}$, achieving linearized inference patterns. Since the product of K and V is element-wise, the recurrent state size is $\mathbb{R}^d$ instead of outer-product state $\mathbb{R}^{d\times d}$.

RWKV [37] leverages decay mechanism on AFT-Simple. Specifically, RWKV improves AFT's position biases with exponential decay $w_{t,i} = -(t-i)w$. The exponential formulation preserves the recurrence property while introducing the position biases.

Element-wise Linear Attention brings strong inference advantage. However, it suffers from the bottleneck of state size, under-performing matrix-based state size. Besides, even though element-wise memory is much fast than outer-product memory, the end-to-end advantage is still marginal since other components occupy more than 95% latency [52] with outer-product memory.

2.4.2. *Multi-Pass Linear Attention*

Attention with Bounded-memory Control considers Linear Attention as a bounded memory module:

$$\begin{aligned} \tilde{K}_n &= \sum_{i=1}^{n} K_i \otimes \phi_i, \quad \tilde{V}_n = \sum_{i=1}^{n} V_i \otimes \phi_i \\ O_n &= \text{softmax}(Q_n \tilde{K}_n^\top) \tilde{V}_n \end{aligned} \tag{11}$$

Where $\tilde{K}_n, \tilde{V}_n$ is the online-updated size-bounded keys and values. In implementation, ABC can be simplified as two-pass Linear Attention.

Gated Slot Attention [77] further introduces GLA into ABC framework [40]. Since $\tilde{K}_n, \tilde{V}_n$ works as an implicit Lienar Attention, GSA improves the update as a gated form:

$$\tilde{K}_n = \text{Diag}(\alpha_n)\tilde{K}_{n-1} + (1-\alpha_n) \otimes K_n, \quad \tilde{V}_n = \text{Diag}(\alpha_n)\tilde{V}_{n-1} + (1-\alpha_n) \otimes V_n \tag{12}$$

Multi-Pass is an effective way to enhance Linear Attention' expressive ability. However, it also brings additional computation overhead, which makes the architecture design as a trade-off between training efficiency and performance.

2.4.3. *Bidirectional Linear Attention*

Bidirectional attention plays an important role in encoder-style architectures such as BERT [13]. The key difference in the linear formulation between unidirectional and bidirectional attention lies in the inference bottleneck and computational pattern. Encoder-only models typically exhibit $O(N^2)$ complexity. Moreover, each token in an encoder-only model has access to global information. As a result, bidirectional linear attention often maintains a constant-length global token pool to reduce complexity while preserving the use of the softmax function.

For instance, Linformer [62] reduces the number of keys and values to a constant length through an additional matrix projection. Luna [34] further extends the Linformer design by encoding the global token pool across model layers.

While bidirectional linear attention is effective for encoder-only architectures, these designs face significant challenges when applied to causal settings as global-pool-based methods tend to be computationally expensive. Consequently, such architectures are not well-suited for Large Language Models.

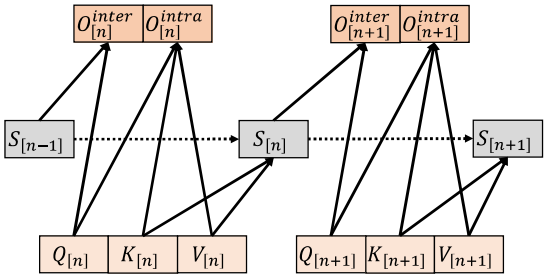
2.5. Hardware Implementation

The Parallel Representation We define the causal linear attention with gated decay as:

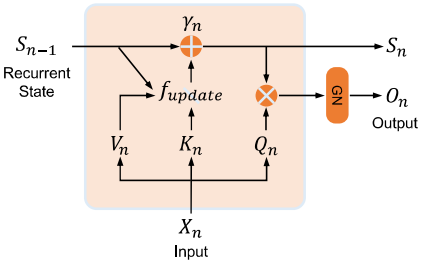
$$\begin{aligned} Q = \phi(XW_Q), \quad K = \phi(XW_K), \quad V = XW_V, \quad \gamma = f_\gamma(X) \\ D_{nm} = \begin{cases} \prod_{i=m+1}^{n} \gamma_i, & n \geq m \\ 0, & n < m \end{cases}, \quad O(X) = \text{LN}((QK^\top \odot D)V) \end{aligned} \tag{13}$$

where $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$, f_γ controls the sharpness of decay. The matrix $D \in \mathbb{R}^{N \times N}$ encodes the causal mask with decay pattern, ensuring uni-directional flow of information.

When the decay is data-independent, $f_\gamma(\cdot) = \text{const} \in (0, 1]$. Note that GroupNorm [63] after Linear Attention is already a compulsory component [52], the explicit devisor of Kernelized Linear Attention in Equation 2 is unessential.



(a) Chunkwise representation.



(b) Recurrent representation.

Figure 2. Dual form of Linear Attention.

The parallel representation is simple and easy to understand but has two shortcomings. First, the parallel form still preserves the $O(N^2)$ complexity, same as softmax Attention. Second, it's complexity increases when representing the ICL-style Linear Attention in Section 2.3.

The Recurrent Representation The parallel formulation above can be equivalently expressed in a recurrent form for step-wise decoding, as illustrated in Figure 2b. At each time step n, the output is computed as:

$$S_n = f_{\text{update}}(S_{n-1}, K_n, V_n), \quad O_n = Q_n S_n$$
$$f_{\text{update}}(S_{n-1}, K_n, V_n) = \begin{cases} \gamma_n S_{n-1} + K_n^\top V_n, & \text{Linear Attention with Decay} \\ \gamma_n S_{n-1} + \eta_n (V_n - S_{n-1} K_n) K_n^\top, & \text{ICL-style Linear Attention} \end{cases} \tag{14}$$

This recurrent formulation enables efficient auto-regressive generation with constant memory by maintaining a single state vector S_n.

While the recurrent representation reduces the computational complexity from $O(N^2)$ to $O(N)$, it incurs substantial memory overhead during training. This is because S_n involves storing outer products of K_n and V_n, which is prohibitively expensive for long sequences. As a result, the recurrent form is typically restricted to the decoding stage.

The Chunkwise Recurrent Representation The chunkwise representation combines the advantages of linear complexity and hardware-friendly parallelism [20, 52]. As shown in Figure 2a, taking decay-style linear attention as an example, given a chunk size B, let $x_{[i]}$ denote the i-th chunk. Define the cumulative decay within a chunk as:

$$\beta_{(i-1)B+j} = \prod_{k=(i-1)B+1}^{(i-1)B+j} \gamma_k, \quad D_{[i]}(j,k) = \begin{cases} \frac{\beta_{(i-1)B+k}}{\beta_{(i-1)B+j}}, & j \le k \\ 0, & \text{otherwise} \end{cases} \tag{15}$$

The chunk-level memory state R_i is computed as:

$$R_i = K_{[i]}^\top (V_{[i]} \odot \frac{\beta_{iB}}{\beta_{[i]}}) + \beta_{iB} R_{i-1} \tag{16}$$

and the output for chunk i is given by:

$$O_{[i]} = (Q_{[i]} K_{[i]}^\top \odot D_{[i]}) V_{[i]} + (Q_{[i]} R_{i-1}) \odot \beta_{[i]} \tag{17}$$

This formulation offers a unified view of recurrence and parallelism: the first term captures intra-chunk dependencies, while the second term propagates inter-chunk memory through a single matrix-vector product. Owing to its efficiency and parallelizability, the chunkwise representation is typically adopted during the training and prefill stages.

For ICL-style linear attention, hardware-friendly chunkwise representations have been developed using Householder transformations [70, 71]. However, for more sophisticated variants such as TTT and Titans, constructing an explicit chunkwise form remains challenging. Instead, these architectures typically rely on large batch sizes for memory updates, effectively simulating chunkwise computation through a fixed hyperparameter.

Kernel-level optimization is essential for achieving high performance. The widely adopted FLA[68] provides a Triton-based implementation for many common Linear Attention modules. Alternatively, custom implementations in CUDA or TileLang[58] are also provided by developer, which can be employed for further acceleration.

3. Sparse Attention

Sparse Attention methods employ the inherit sparse property in attention computation and approximate full attention by

$$\text{Attn}(Q, K, V) = \text{softmax}(QK^{\top}_{[\mathcal{S}]})V_{[\mathcal{S}]} \tag{18}$$

where $\mathcal{S}(t)$ is a subset of indices that query vector $Q(t)$ attend to. Different methods design different selection criteria for $\mathcal{S}(t)$, taking both selection accuracy and hardware efficiency into consideration. Reduce into sub-linear or linear complexity for prefilling or fixed budget for decoding.

3.1. Fixed-pattern Sparse Attention

Several works exploit the structured pattern of token-level sparsity to build Fix-pattern sparsity mask for attention computation.

Local Window Attention Local-window attention confines each query to interact only with neighbouring tokens inside a fixed sliding window w, thus lowering both memory and compute while preserving local context.

Sparse Transformer [8] first applies local-window (row) attention, where w is close to $\sqrt{N}$, then augments it with an extra column attention that summarizes previous locations and propagates information globally. **GPT-3** [5] also adopts a sparse attention pattern similar to that used in Sparse Transformer.

StreamingLLM [64] found that a large amount of attention score is allocated to initial tokens in the input sequence, which they refer as “attention sink". They propose a simple Fixed-pattern Attention which keeps only the sink tokens and sliding window tokens. For instance, given an input sequence with length n, selected token subset $\mathcal{S}(t)$ for query token q_t in StreamingLLM is formulated as

$$\mathcal{S}(t) = \{\, j \mid 0 \le j \le s \ \vee \ t - w \le j \le t \,\},\ \forall t \in [1, n] \tag{19}$$

where s is the sink token size and w is the sliding window size. For better hardware efficiency,

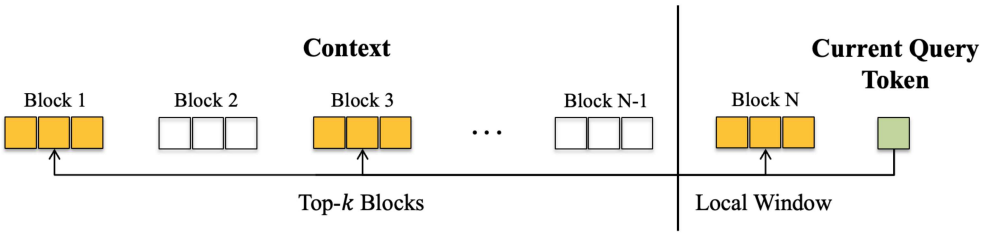


Figure 3. Block-sparse attention: the long sequence is divided into several blocks, and each token attends only to its local window and the top-k related blocks.

StreamingLLM with block granularity [19] keeps sink tokens and local tokens in a block-wise manner, enabling efficient memory loading and computation.

Dilated Attention **LongNet** [14] introduces dilated attention as fixed sparse pattern for long context training and inference. Dilated Attention expands the attention field exponentially as the distance grows, thereby reducing the complexity of attention from $O(n^2)$ to $O(n)$. Specifically, after dividing input along seqnence dimension into segments with length w, dilated sparse index are selected from each segments with an interval r. The selected indices of segment i is:

$$\hat{I}_i = [iw, iw + r, iw + 2r, ..., (i + 1)w - 1] \tag{20}$$

Sparsified segments $Q_{\hat{I}_i}, K_{\hat{I}_i}, V_{\hat{I}_i}$, $i \in \{0, 1, ..., \frac{n}{w}\}$ are fed into the attention in parallel, getting attention output O. Combining attention output of different segment sizes and dillation rates $\{r_i, w_i\}^k$, the final attention is computed as:

$$O = \sum_{i=1}^{K} \alpha_i O|_{r_i, w_i}, \ \alpha_i = \frac{s_i}{\sum_j s_j} \tag{21}$$

where s_i denotes the denominator of the attention softmax for $O|_{r_i, w_i}$. **LogSparse** [27] adopts an exponentially sparse attention scheme in which each position attends to only $logN$ tokens, which can be seen an instance of exponentially dilated attention.

3.2. Block Sparse Attention

Given an input sequence with length n and a block size b, we could divide $Q, K, V \in \mathcal{R}^{n \times d}$ each into $\frac{n}{b}$ blocks, each block sized $b \times d$. The goal is to approximate a block-level Mask $M \in \{0, 1\}^{n/b \times n/b}$ used for selecting critical blocks for computation, as illustrated in Figure 3.

$$\text{Attn}(Q, K, V)_i = \sum_{j=1}^{n/b} M_{ij} \cdot \text{softmax}(Q_i K_j^T) V_j \tag{22}$$

Blockwise selection is crucial to achieve efficient computation on modern GPUs.

3.2.1. Block-Sparse Attention for Prefill

Methods that use Block-Sparse Attention for prefilling approximate Top-K blocks that cover the majority of attention score with high recall ratio, thus reducing the computation complexity of

attention from $O(n^2)$ to $O(K)$.

$$S = \text{softmax}(QK^T - c(1-M)) \\ \min \ |S(M) - S_{dense}| \tag{23}$$

where M is our block-wise sparse mask defined as above, and c is a large constant, such as 1e5, ensuring that less important attention weights approaches zero after the softmax computation. The objective of the block-sparse attention is to achieve greater speedup with minimal overhead while retaining as much of the attention weights as possible.

MInference [21] observed that there are there patterns in attention weights: Streaming (A Shape) Pattern, Vertical-Slash Pattern and Block-Sparse Pattern. It determines the optimal pattern for each attention head offline and dynamically builds sparse indices based on the assigned pattern during inference.

FlexPrefill [25] proposes a context-aware sparse attention mechanism that dynamically adjusts attention patterns and computational budgets in real-time.

XAttention [65] presents a block-sparse attention framework that utilizes antidiagonal scoring to predict the importance of attention blocks, which could efficiently identify and prune non-essential blocks, achieving high sparsity and substantial computational gains.

SpargeAttn [75] also employs block-level sparse attention for prefill, which is done through a double-stage online filtering process: the first stage rapidly predicts the attention map to skip certain matrix multiplications, and the second stage applies a softmax-aware filter to further eliminate unnecessary computations.

3.2.2. Block-Sparse Attention for Decode

Methods that employ Block-Sparse Attention for decoding dynamically select a subset S of K, V vector that contain the most critical tokens for each decoding step, thus reducing memory loading and improving efficiency.

Quest [55] approximate the criticality of each block by calculating an upper bound of attention weights. For block K_i we maintain element-wise Min and Max Key m_i and M_i through

$$m_{i,d} = min(K_{i,d}), \ M_{i,d} = max(K_{i,d}) \tag{24}$$

where $min(\cdot)$ and $max(\cdot)$ are applied element-wise for each dimension d.

Given query q, approximate attention score for block i is given by

$$score_i = \sum_{j=1}^{d} max(q_j \times M_{i,j}, q_j \times m_{i,j}) \tag{25}$$

Then it selects Top-K blocks with highest $score$ as sparse subset S for attention calculation.

$$S = \text{argtopk}(score, k) \tag{26}$$

DoubleSparsity [67] approximates critical tokens efficiently through reducing the matrix multiplication dimension of calculating QK^T product. It first offline calculates outlier channels

in QK^T, denoted as C. Then it selects Top-K tokens with highest approximate attention score $\hat{s}$ as the sparse subset S.

$$Q_{label} = Q_{[C]},\ \hat{s} = Q_{label}K_{label}^T,\ S = \text{argtopk}(\hat{s}, k) \tag{27}$$

ReSA [54] combines training-free block-sparse estimation and GQA sharing, contributing to better efficiency. Besides, ReSA proposes a rectification stage to control the KV cache accumulation error. ReSA shows advantage on long-sequence generation tasks.

3.2.3. Routing-based Block-Sparse Attention

Routing-based Block-Sparse Attention learn the importance of each token block through trainable MLP layers, which act as gating network during inference to select critical blocks.

Learnable Sparsity on Pretrained Models **SeerAttention** [15, 16] train the gating network on pretrained LLMs through a self-distillation manner. To obtain the importance score for each block, it first conduct pooling to Q and K along the sequence dimension, denoted as P_k and P_q. The downsampled Q, K are then passed through a learnbale linear layer W_q and W_k. Matrix multiplied results of projected $W_qP_q(Q)$ and $W_kP_k(K)$ go through the softmax operator as a gating process:

$$score = \text{softmax}((W_qP_q(Q)) \cdot (W_kP_k(K))) \tag{28}$$

The learnable linear layers are trained to align with the 2D maxpooled results of the original LLM through a self-distillation manner. The distillation loss is computed as:

$$gt = \text{MaxPool2D}(\text{softmax}(QK^T)),\ loss = D_{KL}(\,gt||score\,)$$

During inference, the gating score are used to predict block-level sparsity through Top-K or thresholding for sparse computation and efficiency.

Training-aware Sparse Attention **Landmark** [36] proposes using special landmark tokens to represent each block and trains the attention mechanism to directly retrieve Top-K blocks via these landmark tokens. However, it did not experiment on large-scale pretrained models.

MoBA [33] integrate trainable sparse attention into the pretraining stage. It proposes Mixture of Block Attention, which applies the Top-K mechanism from MoE as gating mechanism to decide critical blocks for each query token. The importance score of each block is computed by the inner product between query token q and the mean pooling result of block K_i along the token dimension:

$$s_i =< q, P_{mean}(K_i) > \tag{29}$$

Then the Top-K blocks with highest s score are selected for q in computing attention.

Notably, the Top-K block selection used by MoBA is not differentiable. Therefore, the sparsity pattern is still estimated in a training-free pattern in the pretraining stage, enabling both efficient inference and accelerated training.

NSA [72] introduces a training-aware mix-granularity sparse attention mechanism consisting of three branches $C \in \{\text{cmp}, \text{slc}, \text{win}\}$, which correspond to compression, selection, and sliding

window strategies, respectively. NSA leverages a differentiable compression branch to learn the block selection score.

Combining the three branches, the attention output of NSA is given by

$$o = \sum_{c \in C} g^c \cdot \text{Attn}(q, K^c, V^c), \; g_c \in [0, 1] \tag{30}$$

For the compression branch $c = \text{cmp}$, block i's key $K_i \in \mathcal{R}^{d_k \times b}$ is compressed into a single key $K_i^{\text{cmp}} \in \mathcal{R}^{d_k \times 1}$ through a learnable MLP layer φ. For the selection branch $c = \text{slc}$, Top-K block are selected based on block importance score p, which could be directly obtained from the compression branch.

InfLLM-v2 [57] adopts a training-aware Top-K block sparse attention mechanism similar to MoBA. To Top-K block selection accuracy, it divides blocks into small-granularity kernels with overlap and performs aggregation on kernel importance scores within each block.

3.2.4. System-level Design Choices

Learning-aware Sparse Attention [33, 57, 72] begin to take kernel implementation and efficient execution into consideration. For efficient implementation of Block-Sparse Attention, FlashAttention [10] is used for attention computation in an efficient tilling mechanism, introducing requirements and opportunies for better utilization of hardware resources, including:

- To avoid inconsistency in memory access, in SeerAttention [15] and MInference [21], block size b is typically set to a relatively large value of at least 64.
- To align with the minimal requirement of Grouped Matrix Multiplication instruction on GPU tensor cores, in NSA [72] and InfLLM-v2 [57], the number of K, V heads within a query group are set to at least 16.
- To reduce memory access, NSA [72] and InfLLM-v2 [57] forces sharing of selected blocks among query groups, which is done through conducting pooling on block-level importance score within query groups.

3.3. Clustering Attention

Similar to Block-Sparse Attention, Clustering Attention aims to select the most critical tokens for decoding, but organizes tokens in data structures for better semantic property or sampling efficiency.

RetrievalAttention [30] employs Approximate Nearest Neighbor Search (ANNS) for selecting critical K clusters. To address the challenge of the out-of-distribution nature between query and key vectors in the attention mechanism, it introduces an attention-aware vector search algorithm that adapts to the distribution of query vectors.

ClusterKV [31] select tokens at the granularity of semantic clusters, overcoming the issue of internal fragmentation of page-level retrieval methods such as Quest. After prefilling stage, tokens are clustered through K-means algorithm. The semantic similarity between token i and j are measured through the cosine similarity of key vectors $\mathcal{D}(i, j) = 1 - \frac{<k_i, k_j>}{|k_i| \cdot |k_j|}$. Semantic clusters are represented by their centroids $\mu_1, \mu_2, ..., \mu_C \in \mathcal{R}^d$. At each decoding step, clusters are selected based on the ranking of query token q and centroids μ_i's attention weights, i.e. $q\mu_i^T$.

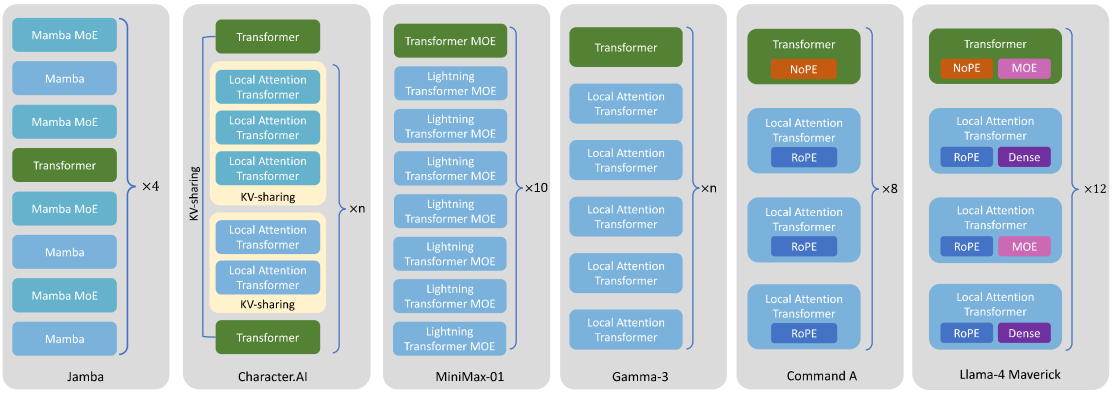


Figure 4. Architecture of different stacked models.

MagicPIG [7] leverages Locality Sensitive Hashing (LSH) sampling to efficiently approximate attention computation. It employs LSH to map similar query and key vectors to the same hash buckets and offloads storage and partial computation to the CPU to address the KV cache bottleneck. It also introduces Oracle Top-K sampling as a better strategy than brute force Top-K.

3.4. Bidirectional Sparse Attention

Bidirectional Sparse Attention builds upon encoder-style architecture, using static pattern or block-level sparsity to accelerate attention computation.

Block sparsity is widely used in bidirectional sparse attention. BigBird [73] uses block-wise random attention, which acts as bridges to shorten indirect paths between tokens. Longformer [4] uses static global-local hybrid attention. It also relies on block-level sparsity with additional global and random links, facilitating structured computation and memory-efficient parallelism.

Clustering-based methods are also used in bidirectional sparse attention. Reformer [24] uses Locality-Sensitive Hashing (LSH) to assign similar tokens to the same bucket. Routing Transformer [48] performs online k-means clustering per layer. ClusterFormer [61] introduces a differentiable clustering module co-trained with downstream objectives. These methods reduce computation by grouping related tokens while maintaining performance through learned adaptability.

4. Pretrained LLM with Efficient Attention

4.1. Pretrained Models with Uniform Efficient Attention

While early explorations of linear attention were often confined to smaller-scale models, recent advancements have demonstrated their successful scalability to the multi-billion parameter range, establishing them as viable and highly efficient alternatives to the standard Transformer. These models, built purely on linear attention or its architectural equivalents like State-Space Models (SSMs) and Recurrent Neural Networks (RNNs), retain their signature inference efficiency even at large scales.

RWKV-based model The RWKV project represents a sustained and influential effort to create a scalable Recurrent Neural Network (RNN) architecture that combines the parallelizable

training of Transformers with the efficient inference of traditional RNNs [37]. For instance, the EAGLE (RWKV-5) series introduced matrix-valued states to increase capacity, while subsequent iterations like Finch (RWKV-6) [38] and Goose (RWKV-7) [39] incorporated dynamic recurrence and expressive state evolution mechanisms (e.g., a delta-rule) to enable more complex, data-dependent state transitions.

Mamba-based model The success of the Mamba architecture [17], with its data-dependent selection mechanism (Section 2.2.2), has spurred a wave of adoption and scaling initiatives from major research labs. **Falcon Mamba** [78] is based on the pure Mamba-based architecture to demonstrate performance competitive with leading Transformer models on a wide range of general-purpose language benchmarks, validating the architecture's viability for such tasks while retaining its signature constant-time inference. Further evidence of this paradigm's potential is provided by **Codestral Mamba** [35], built on the Mamba-2 architecture. While specialized for code generation, it achieves state-of-the-art results on relevant benchmarks and supports a 256k token context, demonstrating the scalability and effectiveness of the SSM approach within a complex and structured domain.

Sparse-based model MiniCPM-4 [57] introduces a two-stage sparse attention mechanism that dynamically selects relevant key-value blocks for each query token based on semantic similarity. MiniCPM-4 leverages InfLLM-v2, a Block Sparse Attention variant to replace standard Attention mechanism. Moreover, a lightweight LogSumExp approximation enables efficient top-k selection, making the method scalable to extremely long sequences. Together, these techniques allow MiniCPM-4 to balance fine-grained contextual awareness with tractable memory and compute requirements, making it a strong candidate for long-context modeling.

4.2. Pretrained Models with Hybrid Efficient Attention

With the increasing demand for efficient long-context modeling and diverse computational paradigms, recent research has extensively explored hybrid attention mechanisms. Such strategies combine global and local attention components, often interleaving specialized layers to balance computational cost and performance. The candidate model architecture illustrations are shown in Figure 4.

Sparse Hybrid Model GPT-3 [5] integrates a hybrid attention mechanism by interleaving dense and locally banded sparse attention layers, inspired by the Sparse Transformer [8]. Dense attention provides full-context modeling, while sparse layers adopt fixed or strided patterns to reduce the number of attended tokens. This design enables GPT-3 to efficiently scale to large model sizes using a fixed context window of 2048 tokens, balancing modeling capacity and computational efficiency.

Linear-Full Hybrid Model Jamba [28] and MiniMax-01 [26] combine linear and full attention layers to achieve an efficient trade-off between throughput and expressiveness. MiniMax-01 employs Lightning Attention across most layers, inserting Softmax-based full attention every eight layers. Jamba adopts a similar ratio, inserting one Transformer layer into every eight-layer Mamba block. Both achieve faster decoding and improved long-sequence performance by limiting the use of computationally intensive full attention.

Local-Full Hybrid Model Gemma 3 [56], Command A [66], and LLaMA-4-Maverick [32] alternate between local and global attention layers, with a shared design philosophy of using global layers sparsely, e.g., every 4–6 layers, to enhance efficiency. While local layers adopt sliding-window patterns, the key difference lies in position encoding strategies. Gemma 3 modulates RoPE base frequencies—assigning 10K for local and 1M for global layers—to better capture long-range dependencies. Command A and LLaMA-4-Maverick mixes RoPE-based local layers with full attention layers that omit positional embeddings entirely, allowing stronger long sequence performance.

Advanced Hybrid Model Character.AI [6] interleaves local attention with sliding windows and sparse global attention layers applied every six layers. Especially, they resue global attention layer's key-value representations across multiple non-adjacent layers. This KV sharing mechanism enables efficient long-context processing with reduced memory and latency overhead.

YOCO [53] and Phi-4-mini-flash [47] adopt a dual-decoder architecture that separates the prefill and generation phases. The Self-Decoder utilizes linear attention mechanisms such as RetNet and Sliding-Window Attention for both prefill and generation, while the Cross-Decoder is activated only during generation. A single-layer global KV cache is used throughout, allowing linear-time prefill and efficient decoding with minimal GPU memory consumption.

In summary, these recent advances underscore the trend toward hybridizing attention mechanisms to achieve balanced performance across varying computational constraints and sequence lengths. Each architecture uniquely contributes insights into effectively combining local detail management with global context integration, thereby providing valuable frameworks for future attention mechanism developments.

5. Outlook

This survey presents a comprehensive overview of efficient attention mechanisms, focusing on their algorithmic foundations, practical implementations, and integration into large-scale pre-trained language models. By categorizing linear and sparse attention into well-defined paradigms, we identify key design principles that enable scalability, computational efficiency, and long-context capability. We also analyze how these mechanisms are deployed in state-of-the-art models, either as standalone architectures or as part of hybrid designs that balance local and global computation.

Looking forward, we highlight several key directions that are expected to shape future research in this area:

Architectural Understanding of Hybrid Models While prior work in linear attention has largely focused on standalone linear architectures, hybrid models are often constructed by combining off-the-shelf linear attention modules with dense or local components. However, it remains unclear whether a stronger linear backbone directly translates to improved hybrid performance. Future work should investigate hybrid models as a distinct architectural class, seeking to understand their composition, interaction effects, and optimization dynamics.

Lossless Sparse Attention and Extended Context Sparse attention remains challenged by a trade-off between accuracy and computational gain. Fully trained sparse models often underperform dense ones, while post-training sparse approximations face limitations due to lack of

end-to-end training. A major research frontier lies in developing sparse attention mechanisms that maintain the expressiveness and accuracy of dense attention, while scaling to much longer contexts. Additionally, the relationship between sparse budget and context length is poorly understood, where fixed top-k schemes may degrade with longer sequences, calling for more adaptive strategies.

Mechanistic Insights into Sparse and Hybrid Attention While empirical studies have repeatedly demonstrated that hybrid attention models can match or exceed dense models using fewer attention computations, the underlying reasons for this effectiveness remain insufficiently explored. Besides, it is especially important to investigate whether the sparsity patterns that work well in synthetic benchmarks generalize to real-world tasks, and to characterize the limits of sparsity-based generalization.

As attention-based models continue to evolve, we expect further convergence between architectural innovation, theoretical insight, and hardware-aware design. We hope this survey provides a solid foundation for future research into efficient, high-performance language modeling systems.

6. Acknowledgment

This work was supported in part by National Key Research and Development Program of China under Grant No. 2020YFA0804503, National Natural Science Foundation of China under Grant No. 62272264, and ByteDance Doubao Large Model Fund Project under Grant No. CT20240909109354.

References

1 Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. xLSTM: Extended long short-term memory. *arXiv preprint arXiv:2405.04517*, 2024.

2 Ali Behrouz, Zeman Li, Praneeth Kacham, Majid Daliri, Yuan Deng, Peilin Zhong, Meisam Razaviyayn, and Vahab Mirrokni. Atlas: Learning to optimally memorize the context at test time. *arXiv preprint arXiv:2505.23735*, 2025.

3 Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2025.

4 Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

5 Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

6 Character.AI. Optimizing ai inference at character.ai. https://research.character.ai/optimizing-inference/, 2024.

7 Zhuoming Chen, Ranajoy Sadhukhan, Zihao Ye, Yang Zhou, Jianyu Zhang, Niklas Nolte, Yuandong Tian, Matthijs Douze, Leon Bottou, Zhihao Jia, and Beidi Chen. Magicpig: Lsh sampling for efficient llm generation, 2024. URL https://arxiv.org/abs/2410.16179.

8 Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse Transformers. *URL https://openai.com/blog/sparse-transformers*, 2019.

9 Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. *arXiv preprint arXiv:2009.14794*, 2020.

10 Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.

11 Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024.

12 Tri Dao, Daniel Y Fu, Khaled K Saab, Armin W Thomas, Atri Rudra, and Christopher Ré. Hungry hungry hippos: Towards language modeling with state space models. *arXiv preprint arXiv:2212.14052*, 2022.

13 Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1423. URL https://www.aclweb.org/anthology/N19-1423.

14 Jiayu Ding, Shuming Ma, Li Dong, Xingxing Zhang, Shaohan Huang, Wenhui Wang, Nanning Zheng, and Furu Wei. Longnet: Scaling transformers to 1,000,000,000 tokens. *arXiv preprint arXiv:2307.02486*, 2023.

15 Yizhao Gao, Zhichen Zeng, Dayou Du, Shijie Cao, Peiyuan Zhou, Jiaxing Qi, Junjie Lai, Hayden Kwok-Hay So, Ting Cao, Fan Yang, et al. Seerattention: Learning intrinsic sparse attention in your llms. *arXiv preprint arXiv:2410.13276*, 2024.

16 Yizhao Gao, Shuming Guo, Shijie Cao, Yuqing Xia, Yu Cheng, Lei Wang, Lingxiao Ma, Yutao Sun, Tianzhu Ye, Li Dong, Hayden Kwok-Hay So, Yu Hua, Ting Cao, Fan Yang, and Mao Yang. Seerattention-r: Sparse attention adaptation for long reasoning, 2025. URL https://arxiv.org/abs/2506.08889.

17 Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

18 Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396*, 2021.

19 Junxian Guo, Haotian Tang, Shang Yang, Zhekai Zhang, Zhijian Liu, and Song Han. Block Sparse Attention. https://github.com/mit-han-lab/Block-Sparse-Attention, 2024.

20 Weizhe Hua, Zihang Dai, Hanxiao Liu, and Quoc Le. Transformer quality in linear time. In *International Conference on Machine Learning*, pages 9099–9117. PMLR, 2022.

21 Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H. Abdi, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=fPBACAbqSN.

22 Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, pages 5156–5165. PMLR, 2020.

23 Tobias Katsch. Gateloop: Fully data-controlled linear recurrence for sequence modeling. *arXiv preprint arXiv:2311.01927*, 2023.

24 Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*, 2020.

25 Xunhao Lai, Jianqiao Lu, Yao Luo, Yiyuan Ma, and Xun Zhou. Flexprefill: A context-aware sparse attention mechanism for efficient long-sequence inference. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=OfjIlbelrT.

26 Aonian Li, Bangwei Gong, Bo Yang, Boji Shan, Chang Liu, Cheng Zhu, Chunhao Zhang, Congchao Guo, Da Chen, Dong Li, et al. Minimax-01: Scaling foundation models with lightning attention. *arXiv preprint arXiv:2501.08313*, 2025.

27 Shiyang Li, Xiaoyong Jin, Yao Xuan, Xiyou Zhou, Wenhu Chen, Yu-Xiang Wang, and Xifeng Yan. Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting. *Advances in neural information processing systems*, 32, 2019.

28 Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, Omri Abend, Raz Alon, Tomer Asida, Amir Bergman, Roman Glozman, Michael Gokhman, Avashalom Manevich,

Nir Ratner, Noam Rozen, Erez Shwartz, Mor Zusman, and Yoav Shoham. Jamba: A hybrid Transformer-Mamba language model. *CoRR*, abs/2403.19887, 2024.

29 Bo Liu, Rui Wang, Lemeng Wu, Yihao Feng, Peter Stone, and Qiang Liu. Longhorn: State space models are amortized online learners. *arXiv preprint arXiv:2407.14207*, 2024.

30 Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Zhenhua Han, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang, Chen Chen, Fan Yang, Yuqing Yang, and Lili Qiu. Retrievalattention: Accelerating long-context llm inference via vector retrieval, 2024. URL https://arxiv.org/abs/2409.10516.

31 Guangda Liu, Chengwei Li, Jieru Zhao, Chenqi Zhang, and Minyi Guo. Clusterkv: Manipulating llm kv cache in semantic space for recallable compression, 2024. URL https://arxiv.org/abs/2412.03213.

32 AI @ Meta Llama Team. Llama 4: Leading intelligence. unrivaled speed and efficiency. https://www.llama.com/models/llama-4/, 2025.

33 Enzhe Lu, Zhejun Jiang, Jingyuan Liu, Yulun Du, Tao Jiang, Chao Hong, Shaowei Liu, Weiran He, Enming Yuan, Yuzhi Wang, Zhiqi Huang, Huan Yuan, Suting Xu, Xinran Xu, Guokun Lai, Yanru Chen, Huabin Zheng, Junjie Yan, Jianlin Su, Yuxin Wu, Neo Y. Zhang, Zhilin Yang, Xinyu Zhou, Mingxing Zhang, and Jiezhong Qiu. Moba: Mixture of block attention for long-context llms, 2025. URL https://arxiv.org/abs/2502.13189.

34 Xuezhe Ma, Xiang Kong, Sinong Wang, Chunting Zhou, Jonathan May, Hao Ma, and Luke Zettlemoyer. Luna: Linear unified nested attention. *Advances in Neural Information Processing Systems*, 34:2441–2453, 2021.

35 Mistral AI Team. Codestral Mamba: A mamba2 language model specialized in code generation, 2024. URL https://mistral.ai/news/codestral-mamba.

36 Amirkeivan Mohtashami and Martin Jaggi. Landmark attention: Random-access infinite context length for transformers. *arXiv preprint arXiv:2305.16300*, 2023.

37 Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Kranthi Kiran GV, Xuzheng He, Haowen Hou, Przemyslaw Kazienko, Jan Kocon, Jiaming Kong, Bartlomiej Koptyra, Hayden Lau, Krishna Sri Ipsit Mantri, Ferdinand Mom, Atsushi Saito, Xiangru Tang, Bolun Wang, Johan S. Wind, Stansilaw Wozniak, Ruichong Zhang, Zhenyuan Zhang, Qihang Zhao, Peng Zhou, Jian Zhu, and Rui-Jie Zhu. Rwkv: Reinventing rnns for the transformer era, 2023.

38 Bo Peng, Daniel Goldstein, Quentin Anthony, Alon Albalak, Eric Alcaide, Stella Biderman, Eugene Cheah, Teddy Ferdinan, Haowen Hou, Przemysław Kazienko, et al. Eagle and finch: Rwkv with matrix-valued states and dynamic recurrence. *arXiv preprint arXiv:2404.05892*, 3, 2024.

39 Bo Peng, Ruichong Zhang, Daniel Goldstein, Eric Alcaide, Xingjian Du, Haowen Hou, Jiaju Lin, Jiaxing Liu, Janna Lu, William Merrill, Guangyu Song, Kaifeng Tan, Saiteja Utpala, Nathan Wilce, Johan S. Wind, Tianyi Wu, Daniel Wuttke, and Christian Zhou-Zheng. Rwkv-7 "goose" with expressive dynamic state evolution, 2025. URL https://arxiv.org/abs/2503.14456.

40 Hao Peng, Jungo Kasai, Nikolaos Pappas, Dani Yogatama, Zhaofeng Wu, Lingpeng Kong, Roy Schwartz, and Noah A Smith. Abc: Attention with bounded-memory control. *arXiv preprint arXiv:2110.02488*, 2021.

41 Hao Peng, Nikolaos Pappas, Dani Yogatama, Roy Schwartz, Noah A Smith, and Lingpeng Kong. Random feature attention. *arXiv preprint arXiv:2103.02143*, 2021.

42 Zhen Qin, Weixuan Sun, Hui Deng, Dongxu Li, Yunshen Wei, Baohong Lv, Junjie Yan, Lingpeng Kong, and Yiran Zhong. cosformer: Rethinking softmax in attention. *arXiv preprint arXiv:2202.08791*, 2022.

43 Zhen Qin, Dong Li, Weigao Sun, Weixuan Sun, Xuyang Shen, Xiaodong Han, Yunshen Wei, Baohong Lv, Xiao Luo, Yu Qiao, et al. Transnormerllm: A faster and better large language model with improved transnormer. *arXiv preprint arXiv:2307.14995*, 2023.

44 Zhen Qin, Songlin Yang, and Yiran Zhong. Hierarchically gated recurrent neural network for sequence modeling. *Advances in Neural Information Processing Systems*, 36:33202–33221, 2023.

45 Zhen Qin, Weigao Sun, Dong Li, Xuyang Shen, Weixuan Sun, and Yiran Zhong. Various lengths, constant speed: Efficient language modeling with lightning attention. *arXiv preprint arXiv:2405.17381*, 2024.

46 Zhen Qin, Songlin Yang, Weixuan Sun, Xuyang Shen, Dong Li, Weigao Sun, and Yiran Zhong. Hgrn2: Gated linear rnns with state expansion. *arXiv preprint arXiv:2404.07904*, 2024.

47 Liliang Ren, Congcong Chen, Haoran Xu, Young Jin Kim, Adam Atkinson, Zheng Zhan, Jiankai Sun, Baolin Peng, Liyuan Liu, Shuohang Wang, et al. Decoder-hybrid-decoder architecture for efficient reasoning with long generation. *arXiv preprint arXiv:2507.06607*, 2025.

48 Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers, 2020. URL https://arxiv.org/abs/2003.05997.

49 Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *International conference on machine learning*, pages 9355–9366. PMLR, 2021.

50 Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *International Conference on Machine Learning (ICML)*, 2020.

51 Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, et al. Learning to (learn at test time): Rnns with expressive hidden states. *arXiv preprint arXiv:2407.04620*, 2024.

52 Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

53 Yutao Sun, Li Dong, Yi Zhu, Shaohan Huang, Wenhui Wang, Shuming Ma, Quanlu Zhang, Jianyong Wang, and Furu Wei. You only cache once: Decoder-decoder architectures for language models, 2024. URL https://arxiv.org/abs/2405.05254.

54 Yutao Sun, Tianzhu Ye, Li Dong, Yuqing Xia, Jian Chen, Yizhao Gao, Shijie Cao, Jianyong Wang, and Furu Wei. Rectified sparse attention. *arXiv preprint arXiv:2506.04108*, 2025.

55 Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context llm inference, 2024. URL https://arxiv.org/abs/2406.10774.

56 Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, Gaël Liu, Francesco Visin, Kathleen Kenealy, Lucas Beyer, Xiaohai Zhai, Anton Tsitsulin, Robert Busa-Fekete, Alex Feng, Noveen Sachdeva, Benjamin Coleman, Yi Gao, Basil Mustafa, Iain Barr, Emilio Parisotto, David Tian, Matan Eyal, Colin Cherry, Jan-Thorsten Peter, Danila Sinopalnikov, Surya Bhupatiraju, Rishabh Agarwal, Mehran Kazemi, Dan Malkin, Ravin Kumar, David Vilar, Idan Brusilovsky, Jiaming Luo, Andreas Steiner, Abe Friesen, Abhanshu Sharma, Abheesht Sharma, Adi Mayrav Gilady, Adrian Goedeckemeyer, Alaa Saade, Alex Feng, Alexander Kolesnikov, Alexei Bendebury, Alvin Abdagic, Amit Vadi, András György, André Susano Pinto, Anil Das, Ankur Bapna, Antoine Miech, Antoine Yang, Antonia Paterson, Ashish Shenoy, Ayan Chakrabarti, Bilal Piot, Bo Wu, Bobak Shahriari, Bryce Petrini, Charlie Chen, Charline Le Lan, Christopher A. Choquette-Choo, CJ Carey, Cormac Brick, Daniel Deutsch, Danielle Eisenbud, Dee Cattle, Derek Cheng, Dimitris Paparas, Divyashree Shivakumar Sreepathihalli, Doug Reid, Dustin Tran, Dustin Zelle, Eric Noland, Erwin Huizenga, Eugene Kharitonov, Frederick Liu, Gagik Amirkhanyan, Glenn Cameron, Hadi Hashemi, Hanna Klimczak-Plucińska, Harman Singh, Harsh Mehta, Harshal Tushar Lehri, Hussein Hazimeh, Ian Ballantyne, Idan Szpektor, Ivan Nardini, Jean Pouget-Abadie, Jetha Chan, Joe Stanton, John Wieting, Jonathan Lai, Jordi Orbay, Joseph Fernandez, Josh Newlan, Ju yeong Ji, Jyotinder Singh, Kat Black, Kathy Yu, Kevin Hui, Kiran Vodrahalli, Klaus Greff, Linhai Qiu, Marcella Valentine, Marina Coelho, Marvin Ritter, Matt Hoffman, Matthew Watson, Mayank Chaturvedi, Michael Moynihan, Min Ma, Nabila Babar, Natasha Noy, Nathan Byrd, Nick Roy, Nikola Momchev, Nilay Chauhan, Noveen Sachdeva, Oskar Bunyan, Pankil Botarda, Paul Caron, Paul Kishan Rubenstein, Phil Culliton, Philipp Schmid, Pier Giuseppe Sessa, Pingmei Xu, Piotr Stanczyk, Pouya Tafti, Rakesh Shivanna, Renjie Wu, Renke Pan, Reza Rokni, Rob Willoughby, Rohith Vallu, Ryan Mullins, Sammy Jerome, Sara Smoot, Sertan Girgin, Shariq Iqbal, Shashir Reddy, Shruti Sheth, Siim Põder, Sijal Bhatnagar, Sindhu Raghuram Panyam, Sivan Eiger, Susan Zhang, Tianqi Liu, Trevor Yacovone, Tyler Liechty, Uday Kalra, Utku Evci, Vedant Misra, Vincent Roseberry, Vlad Feinberg, Vlad Kolesnikov, Woohyun Han, Woosuk Kwon, Xi Chen, Yinlam Chow, Yuvein Zhu, Zichuan Wei, Zoltan Egyed, Victor Cotruta, Minh Giang, Phoebe Kirk, Anand Rao, Kat Black, Nabila Babar, Jessica Lo, Erica Moreira, Luiz Gustavo Martins, Omar Sanseviero, Lucas Gonzalez, Zach Gleicher, Tris Warkentin, Vahab Mirrokni, Evan Senter, Eli Collins, Joelle Barral, Zoubin Ghahramani, Raia Hadsell, Yossi Matias, D. Sculley, Slav Petrov, Noah Fiedel, Noam Shazeer, Oriol Vinyals, Jeff Dean, Demis Hassabis, Koray Kavukcuoglu, Clement Farabet, Elena Buchatskaya, Jean-Baptiste Alayrac, Rohan Anil, Dmitry, Lepikhin, Sebastian Borgeaud, Olivier Bachem, Armand Joulin, Alek Andreev, Cassidy Hardin, Robert Dadashi, and Léonard Hussenot. Gemma 3 technical report, 2025. URL https://arxiv.org/abs/2503.19786.

57 MiniCPM Team, Chaojun Xiao, Yuxuan Li, Xu Han, Yuzhuo Bai, Jie Cai, Haotian Chen, Wentong Chen, Xin Cong, Ganqu Cui, Ning Ding, Shengdan Fan, Yewei Fang, Zixuan Fu, Wenyu Guan, Yitong Guan, Junshao Guo, Yufeng Han, Bingxiang He, Yuxiang Huang, Cunliang Kong, Qiuzuo Li, Siyuan Li, Wenhao Li, Yanghao Li, Yishan Li, Zhen Li, Dan Liu, Biyuan Lin, Yankai Lin, Xiang Long, Quanyu Lu, Yaxi Lu, Peiyan Luo, Hongya Lyu, Litu Ou, Yinxu Pan, Zekai Qu, Qundong Shi, Zijun Song, Jiayuan Su, Zhou Su, Ao Sun, Xianghui Sun,

Peijun Tang, Fangzheng Wang, Feng Wang, Shuo Wang, Yudong Wang, Yesai Wu, Zhenyu Xiao, Jie Xie, Zihao Xie, Yukun Yan, Jiarui Yuan, Kaihuo Zhang, Lei Zhang, Linyue Zhang, Xueren Zhang, Yudi Zhang, Hengyu Zhao, Weilin Zhao, Weilun Zhao, Yuanqian Zhao, Zhi Zheng, Ge Zhou, Jie Zhou, Wei Zhou, Zihan Zhou, Zixuan Zhou, Zhiyuan Liu, Guoyang Zeng, Chao Jia, Dahai Li, and Maosong Sun. Minicpm4: Ultra-efficient llms on end devices, 2025. URL https://arxiv.org/abs/2506.07900.

58 Tile.ai. Tilelang: A tensor dsl for compiling sparse and dense attention. https://github.com/tile-ai/tilelang, 2024. Accessed: 2025-05-06.

59 Yao-Hung Hubert Tsai, Shaojie Bai, Makoto Yamada, Louis-Philippe Morency, and Ruslan Salakhutdinov. Transformer dissection: a unified understanding of transformer's attention via the lens of kernel. *arXiv preprint arXiv:1908.11775*, 2019.

60 Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, 4-9 December 2017, Long Beach, CA, USA*, pages 6000–6010, 2017.

61 Ningning Wang, Guobing Gan, Peng Zhang, Shuai Zhang, Junqiu Wei, Qun Liu, and Xin Jiang. ClusterFormer: Neural clustering attention for efficient and effective transformer. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio, editors, *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2390–2402, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.acl-long.170. URL https://aclanthology.org/2022.acl-long.170/.

62 Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

63 Yuxin Wu and Kaiming He. Group normalization. In *Proceedings of the European conference on computer vision (ECCV)*, pages 3–19, 2018.

64 Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.

65 Ruyi Xu, Guangxuan Xiao, Haofeng Huang, Junxian Guo, and Song Han. Xattention: Block sparse attention with antidiagonal scoring. *arXiv*, 2025.

66 Bowen Yang, Bharat Venkitesh, Dwarak Talupuru, Hangyu Lin, David Cairuz, Phil Blunsom, and Acyr Locatelli. Rope to nope and back again: A new hybrid attention strategy. *arXiv preprint arXiv:2501.18795*, 2025.

67 Shuo Yang, Ying Sheng, Joseph E. Gonzalez, Ion Stoica, and Lianmin Zheng. Post-training sparse attention with double sparsity, 2024. URL https://arxiv.org/abs/2408.07092.

68 Songlin Yang and Yu Zhang. FLA: A Triton-based library for hardware-efficient implementations of linear attention mechanism. https://github.com/sustcsonglin/flash-linear-attention, 2024.

69 Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. *arXiv preprint arXiv:2312.06635*, 2023.

70 Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. *arXiv preprint arXiv:2412.06464*, 2024.

71 Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. In *Neural Information Processing Systems (NeurIPS)*, 2024.

72 Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention, 2025. URL https://arxiv.org/abs/2502.11089.

73 Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 33: 17283–17297, 2020.

74 Shuangfei Zhai, Walter Talbott, Nitish Srivastava, Chen Huang, Hanlin Goh, Ruixiang Zhang, and Josh Susskind. An attention free transformer. *arXiv preprint arXiv:2105.14103*, 2021.

75 Jintao Zhang, Chendong Xiang, Haofeng Huang, Jia Wei, Haocheng Xi, Jun Zhu, and Jianfei Chen. Spargeattn: Accurate sparse attention accelerating any model inference, 2025. URL https://arxiv.org/abs/2502.18137.

76 Michael Zhang, Kush Bhatia, Hermann Kumbong, and Christopher Ré. The hedgehog & the porcupine: Expressive linear attentions with softmax mimicry. *arXiv preprint arXiv:2402.04347*, 2024.

77 Yu Zhang, Songlin Yang, Ruijie Zhu, Yue Zhang, Leyang Cui, Yiqiao Wang, Bolun Wang, Freda Shi, Bailin Wang, Wei Bi, et al. Gated slot attention for efficient linear-time sequence modeling. *arXiv preprint arXiv:2409.07146*, 2024.

78 Jingwei Zuo, Maksim Velikanov, Dhia Eddine Rhaiem, Ilyas Chahed, Younes Belkada, Guillaume Kunsch, and Hakim Hacid. Falcon mamba: The first competitive attention-free 7b language model, 2024. URL https://arxiv.org/abs/2410.05355.